

CPSC 340 Assignment 4

Important: Submission Format [5 points]

Please make sure to follow the submission instructions posted on the course website. We will deduct marks if the submission format is incorrect, or if you're not using L^AT_EX – at least these 5 points, more for egregious issues.

1 Convex Functions [15 points]

Recall that convex loss functions are typically easier to minimize than non-convex functions, so it's important to be able to identify whether a function is convex.

Show that the following functions are convex:

1. $f(w) = \alpha w^2 - \beta w + \gamma$ with $w \in \mathbb{R}, \alpha \geq 0, \beta \in \mathbb{R}, \gamma \in \mathbb{R}$ (1D quadratic).
2. $f(w) = -\log(\alpha w)$ with $\alpha > 0$ and $w > 0$ (“negative logarithm”).
3. $f(w) = \|Xw - y\|_2 + \frac{\lambda}{2}\|w\|_1$ with $w \in \mathbb{R}^d, \lambda \geq 0$ (L1-regularized linear regression).
4. $f(w) = \sum_{i=1}^n \log(1 + \exp(-y_i w^T x_i))$ with $w \in \mathbb{R}^d$ (logistic regression).
5. $f(w) = \sum_{i=1}^n [\max\{0, |w^T x_i - y_i|\} - \epsilon] + \frac{\lambda}{2}\|w\|_2^2$ with $w \in \mathbb{R}^d, \epsilon \geq 0, \lambda \geq 0$ (support vector regression).

General hint: for the first two you can check that the second derivative is non-negative since they are one-dimensional. For the last 3, it's easier to use some of the results regarding how combining convex functions can yield convex functions; which can be found in the lecture slides.

Hint for part 4 (logistic regression): this function may at first seem non-convex since it contains $\log(z)$ and $\log$ is concave, but note that $\log(\exp(z)) = z$ is convex despite containing a $\log$. To show convexity, you can reduce the problem to showing that $\log(1 + \exp(z))$ is convex, which can be done by computing the second derivative. It may simplify matters to note that $\frac{\exp(z)}{1 + \exp(z)} = \frac{1}{1 + \exp(-z)}$.

2 Logistic Regression with Sparse Regularization [30 points]

If you run `python main.py 2`, it will:

1. Load a binary classification dataset containing a training and a validation set.
2. Standardize the columns of `X`, and add a bias variable (in `utils.load_dataset`).
3. Apply the same transformation to `Xvalidate` (in `utils.load_dataset`).
4. Fit a logistic regression model.
5. Report the number of features selected by the model (number of non-zero regression weights).
6. Report the error on the validation set.

Logistic regression does reasonably well on this dataset, but it uses all the features (even though only the prime-numbered features are relevant) and the validation error is above the minimum achievable for this model (which is 1 percent, if you have enough data and know which features are relevant). In this question, you will modify this demo to use different forms of regularization to improve on these aspects.

Note: your results may vary slightly, depending on your software versions, the exact order you do floating-point operations in, and so on.

2.1 L2-Regularization [5 points]

In `linear_models.py`, you will find a class named `LinearClassifier` that defines the fitting and prediction behaviour of a logistic regression classifier. As with ordinary least squares linear regression, the particular choice of a function object (`fun_obj`) and an optimizer (`optimizer`) will determine the properties of your output model. Your task is to implement a logistic regression classifier that uses L2-regularization on its weights. Go to `fun_obj.py` and complete the `LogisticRegressionLossL2` class. This class' constructor takes an input parameter λ , the L2 regularization weight. Specifically, while `LogisticRegressionLoss` computes

$$f(w) = \sum_{i=1}^n \log(1 + \exp(-y_i w^T x_i)),$$

your new class `LogisticRegressionLossL2` should compute

$$f(w) = \sum_{i=1}^n [\log(1 + \exp(-y_i w^T x_i))] + \frac{\lambda}{2} \|w\|^2.$$

and its gradient. Submit your function object code. Using this new code with $\lambda = 1$, report how the following quantities change: (1) the training (classification) error, (2) the validation (classification) error, (3) the number of features used, and (4) the number of gradient descent iterations.

Note: as you may have noticed, `lambda` is a special keyword in Python, so we can't use it as a variable name. Some alternative options: `lammy` (what Mike's niece calls her toy stuffed lamb), `lamda`, `reg_wt`, λ if you feel like typing it, the sheep emoji¹,

2.2 L1-Regularization and Regularization Path [5 points]

L1-regularized logistic regression classifier has the following objective function:

$$f(w) = \sum_{i=1}^n [\log(1 + \exp(-y_i w^T x_i))] + \lambda \|w\|_1.$$

¹Harder to insert in L^AT_EX than you'd like; turns out there are some drawbacks to using software written in 1978.

Because the L1 norm isn't differentiable when any elements of w are 0 – and that's *exactly what we want to get* – standard gradient descent isn't going to work well on this objective. There is, though, a similar approach called *proximal gradient descent* that does work here.²

This is implemented for you in the `GradientDescentLineSearchProxL1` class inside `optimizers.py`. Note that to use it, you *don't include the L1 penalty in your loss function object*; the optimizer handles that itself.

Write and submit code to instantiate `LinearClassifier` with the correct function object and optimizer for L1-regularization. Using this linear model, obtain solutions for L1-regularized logistic regression with $\lambda = 0.01$, $\lambda = 0.1$, $\lambda = 1$, $\lambda = 10$. Report the following quantities per each value of λ : (1) the training error, (2) the validation error, (3) the number of features used, and (4) the number of gradient descent iterations.

2.3 L0 Regularization [8 points]

The class `LogisticRegressionLossL0` in `fun_obj.py` contains part of the code needed to implement the *forward selection* algorithm, which approximates the solution with L0-regularization,

$$f(w) = \sum_{i=1}^n [\log(1 + \exp(-y_i w^T x_i))] + \lambda \|w\|_0.$$

The class `LinearClassifierForwardSel` in `linear_models.py` will use a loss function object and an optimizer to perform a forward selection to approximate the best feature set. The `for` loop in its `fit()` method is missing the part where we fit the model using the subset `selected_new`, then compute the score and updates the `min_loss` and `best_feature`. Modify the `for` loop in this code so that it fits the model using only the features `selected_new`, computes the score above using these features, and updates the variables `min_loss` and `best_feature`, as well as `self.total_evals`. **Hand in your updated code.** Using this new code with $\lambda = 1$, report the training error, validation error, number of features selected, and total optimization steps.

²Here's an explanation, as **bonus content** you don't need to understand.

(Feel free to delete this overly long footnote from your answers file, if you want.)

For the explanation to make sense, it'll help to first re-frame gradient descent in the following way: to take a step from w^t while trying to minimize an objective function f , we first make a *quadratic approximation* to f around the point w^t of the form

$$\tilde{f}^t(w) = f(w^t) + [\nabla f(w^t)]^T (w - w^t) + \frac{1}{2\alpha^t} \|w - w^t\|^2.$$

This is like taking a Taylor expansion of f , but instead of using the expensive-to-compute Hessian $\nabla^2 f$, we just use $\frac{1}{\alpha^t} I$. Then we minimize that approximation to find our next step: $w^{t+1} = \arg \min_w \tilde{f}^t(w)$, which if you do out the math ends up being exactly our old friend $w^{t+1} = w^t - \alpha^t \nabla f(w^t)$.³

In proximal gradient descent, our objective $f(w)$ is of the form $g(w) + h(w)$, where g is a smooth function (e.g. the logistic regression loss) but h might not be differentiable. Then the idea of proximal gradient descent is that we do the quadratic approximation for g but just leave h alone:

$$\begin{aligned} w^{t+1} &= \arg \min_w g(w^t) + [\nabla g(w^t)]^T (w - w^t) + \frac{1}{2\alpha^t} \|w - w^t\|^2 + h(w) \\ &= \arg \min_w \frac{1}{2\alpha^t} \|w - (w^t - \alpha^t \nabla g(w^t))\|^2 + h(w), \end{aligned} \quad (\text{prox})$$

an optimization problem trying to trade off being close to the gradient descent update (first term) with keeping h small (second).

As long as you can compute $\nabla g(w)$, this problem otherwise *doesn't depend on g at all*: you can just run the gradient descent update based on g then plug that into the “prox update” (prox). For many important functions h , this is available in closed form. For L1 regularization we have $h(w) = \lambda \|w\|_1$, and it turns out that the solution is the “soft-thresholding” function, given elementwise by

$$\left[\arg \min_w \frac{1}{2\alpha} \|w - z\|^2 + \lambda \|w\|_1 \right]_i = \begin{cases} z_i - \alpha\lambda & \text{if } z_i > \alpha\lambda \\ 0 & \text{if } |z_i| \leq \alpha\lambda \\ z_i + \alpha\lambda & \text{if } z_i < -\alpha\lambda \end{cases}.$$

³Incidentally, using the real Hessian here is called Newton's method. This is a much better approximation to f , and so the update steps it takes can be much better than gradient descent, causing it to converge in many fewer iterations. But each of these iterations is much more computationally expensive, since we need to compute and solve a linear system with the $d \times d$ Hessian. In ML settings it's often too computationally expensive to run.

Note that the code differs slightly from what we discussed in class, since we're hard-coding that we include the first (bias) variable. Also, note that for this particular case using the L0-norm with $\lambda = 1$ is using the Akaike Information Criterion (AIC) for variable selection.

Also note that, for numerical reasons, your answers may vary depending on exactly what system and package versions you are using. That is fine.

2.4 Discussion [4 points]

In a short paragraph, briefly discuss your results from the above. How do the different forms of regularization compare with each other? Can you provide some intuition for your results? No need to write a long essay, please!

2.5 $L_{\frac{1}{2}}$ regularization [8 points]

Previously we've considered L2- and L1- regularization which use the L2 and L1 norms respectively. Now consider least squares linear regression with " $L_{\frac{1}{2}}$ regularization" (in quotation marks because the " $L_{\frac{1}{2}}$ norm" is not a true norm):

$$f(w) = \frac{1}{2} \sum_{i=1}^n (w^T x_i - y_i)^2 + \lambda \sum_{j=1}^d |w_j|^{1/2}.$$

Let's consider the case of $d = 1$ and assume there is no intercept term being used, so the loss simplifies to

$$f(w) = \frac{1}{2} \sum_{i=1}^n (wx_i - y_i)^2 + \lambda \sqrt{|w|}.$$

Finally, let's assume the very special case of $n = 2$, where our 2 data points are $(x_1, y_1) = (1, 2)$ and $(x_2, y_2) = (0, 1)$.

1. Plug in the dataset values and write the loss in a simplified form, without a $\sum$.
2. If $\lambda = 0$, what is the solution, i.e. $\arg \min_w f(w)$?
3. If $\lambda \rightarrow \infty$, what is the solution, i.e., $\arg \min_w f(w)$?
4. Plot $f(w)$ when $\lambda = 1$. What is $\arg \min_w f(w)$ when $\lambda = 1$? Answer to one decimal place if appropriate. (For the plotting questions, you can use `matplotlib` or any graphing software, such as <https://www.desmos.com>.)
5. Plot $f(w)$ when $\lambda = 10$. What is $\arg \min_w f(w)$ when $\lambda = 10$? Answer to one decimal place if appropriate.
6. Does $L_{\frac{1}{2}}$ regularization behave more like L1 regularization or L2 regularization when it comes to performing feature selection? Briefly justify your answer.
7. Is least squares with $L_{\frac{1}{2}}$ regularization a convex optimization problem? Briefly justify your answer.

3 Multi-Class Logistic Regression [32 points]

If you run `python main.py 3` the code loads a multi-class classification dataset with $y_i \in \{0, 1, 2, 3, 4\}$ and fits a “one-vs-all” classification model using least squares, then reports the validation error and shows a plot of the data/classifier. The performance on the validation set is ok, but could be much better. For example, this classifier never even predicts that examples will be in classes 0 or 4.

3.1 Softmax Classification, toy example [4 points]

Linear classifiers make their decisions by finding the class label c maximizing the quantity $w_c^T x_i$, so we want to train the model to make $w_{y_i}^T x_i$ larger than $w_{c'}^T x_i$ for all the classes c' that are not y_i . Here c' is a possible label and $w_{c'}$ is row c' of W . Similarly, y_i is the training label, w_{y_i} is row y_i of W , and in this setting we are assuming a discrete label $y_i \in \{1, 2, \dots, k\}$. Before we move on to implementing the softmax classifier to fix the issues raised in the introduction, let's work through a toy example:

Consider the dataset below, which has $n = 10$ training examples, $d = 2$ features, and $k = 3$ classes:

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \\ 1 & 0 \\ 1 & 1 \\ 1 & 1 \\ 0 & 0 \\ 1 & 0 \\ 1 & 0 \\ 1 & 1 \\ 1 & 0 \end{bmatrix}, \quad y = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 2 \\ 2 \\ 2 \\ 2 \\ 3 \\ 3 \\ 3 \end{bmatrix}.$$

Suppose that you want to classify the following test example:

$$\tilde{x} = \begin{bmatrix} 1 & 1 \end{bmatrix}.$$

Suppose we fit a multi-class linear classifier using the softmax loss, and we obtain the following weight matrix:

$$W = \begin{bmatrix} +4 & -1 \\ +2 & -2 \\ +3 & -1 \end{bmatrix}$$

Under this model, what class label would we assign to the test example? (Show your work.)

3.2 One-vs-all Logistic Regression [7 points]

Using the squared error on this problem hurts performance because it has “bad errors” (the model gets penalized if it classifies examples “too correctly”). In `linear_models.py`, complete the class named `LinearClassifierOneVsAll` that replaces the squared loss in the one-vs-all model with the logistic loss. [Hand in the code and report the validation error.](#)

3.3 Softmax Classifier Gradient [7 points]

Using a one-vs-all classifier can hurt performance because the classifiers are fit independently, so there is no attempt to calibrate the columns of the matrix W . As we discussed in lecture, an alternative to this independent model is to use the softmax loss, which is given by

$$f(W) = \sum_{i=1}^n \left[-w_{y_i}^T x_i + \log \left(\sum_{c'=1}^k \exp(w_{c'}^T x_i) \right) \right],$$

Show that the partial derivatives of this function, which make up its gradient, are given by the following expression:

$$\frac{\partial f}{\partial W_{cj}} = \sum_{i=1}^n x_{ij} [p(y_i = c | W, x_i) - \mathbb{1}(y_i = c)],$$

where...

- $\mathbb{1}(y_i = c)$ is the indicator function (it is 1 when $y_i = c$ and 0 otherwise)
- $p(y_i = c | W, x_i)$ is the predicted probability of example i being class c , defined as

$$p(y_i = c | W, x_i) = \frac{\exp(w_c^T x_i)}{\sum_{c'=1}^k \exp(w_{c'}^T x_i)}$$

3.4 Softmax Classifier Implementation [8 points]

Inside `linear_models.py`, you will find the class `MulticlassLinearClassifier`, which fits W using the softmax loss from the previous section instead of fitting k independent classifiers. As with other linear models, you must implement a function object class in `fun_obj.py`. Find the class named `SoftmaxLoss`. Complete these classes and their methods. [Submit your code and report the validation error.](#)

Hint: You may want to use `check_correctness()` to check that your implementation of the gradient is correct.

Hint: With softmax classification, our parameters live in a matrix W instead of a vector w . However, most optimization routines (like `scipy.optimize.minimize` or our `optimizers.py`) are set up to optimize with respect to a vector of parameters. The standard approach is to “flatten” the matrix W into a vector (of length kd , in this case) before passing it into the optimizer. On the other hand, it’s inconvenient to work with the flattened form everywhere in the code; intuitively, we think of it as a matrix W and our code will be more readable if the data structure reflects our thinking. Thus, the approach we recommend is to reshape the parameters back and forth as needed. The skeleton code of `SoftmaxLoss` already has lines reshaping the input vector w into a $k \times d$ matrix using `np.reshape`. You can then compute the gradient using sane, readable code with the W matrix inside `evaluate()`. You’ll end up with a gradient that’s also a matrix: one partial derivative per element of W . Right at the end of `evaluate()`, you can flatten this gradient matrix into a vector using `g.reshape(-1)`. If you do this, the optimizer will be sending in a vector of parameters to `SoftmaxLoss`, and receiving a gradient vector back out, which is the interface it wants – and your `SoftmaxLoss` code will be much more readable, too. You may need to do a bit more reshaping elsewhere, but this is the key piece.

Hint: A naïve implementation of `SoftmaxLoss.evaluate()` might involve many for-loops, which is fine as long as the function and gradient calculations are correct. However, this method might take a very long time! This speed bottleneck is one of Python’s shortcomings, which can be addressed by employing pre-computing and lots of vectorized operations. However, it can be difficult to convert your written solutions of f and g into vectorized forms, so you should prioritize getting the implementation to work correctly first. One reasonable path is to first make a correct function and gradient implementation with lots of loops, then (if you want) pulling bits out of the loops into meaningful variables, and then thinking about how you can compute each of the variables in a vectorized way.

3.5 Comparison with scikit-learn [2 points]

[Compare your results \(training error and validation error for both one-vs-all and softmax\) with scikit-learn’s `LogisticRegression`](#), which can also handle multi-class problems. For one-vs-all, set `multi_class='ovr'`; for softmax, set `multi_class='multinomial'`. Since your comparison code above isn’t using regularization,

set `penalty='none'`. Again, set `fit_intercept` to `False` for the same reason as above (there is already a column of 1's added to the data set).

3.6 Cost of Multi-Class Logistic Regression [4 points]

Assume that we have

- n training examples.
- d features.
- k classes.
- t testing examples.
- T iterations of gradient descent for training.

Also assume that we take X and form new features Z using Gaussian RBFs as a non-linear feature transformation.

1. In $O()$ notation, what is the cost of training the softmax classifier with gradient descent?
2. What is the cost of classifying the t test examples?

Hint: you'll need to take into account the cost of forming the basis at training (Z) and test ($\tilde{Z}$) time. It will be helpful to think of the dimensions of all the various matrices.

4 Very-Short Answer Questions [18 points]

Answer each of the following questions in a sentence or two.

1. Suppose that a client wants you to identify the set of “relevant” factors that help prediction. Should you promise them that you can do this?
2. What is a setting where you would use the L1-loss, and what is a setting where you would use L1-regularization?
3. Among L0-regularization, L1-regularization, and L2-regularization: which yield convex objectives? Which yield unique solutions? Which yield sparse solutions?
4. What is the effect of λ in L1-regularization on the sparsity level of the solution? What is the effect of λ on the two parts of the fundamental trade-off?
5. Suppose you have a feature selection method that tends not to generate false positives, but has many false negatives (it misses relevant variables). Describe an ensemble method for feature selection that could improve the performance of this method.
6. Suppose a binary classification dataset has 3 features. If this dataset is “linearly separable”, what does this precisely mean in three-dimensional space?
7. When searching for a good w for a linear classifier, why do we use the logistic loss instead of just minimizing the number of classification errors?
8. What is a disadvantage of using the perceptron algorithm to fit a linear classifier?
9. How does the hyper-parameter σ affect the shape of the Gaussian RBFs bumps? How does it affect the fundamental tradeoff?